

Corso di Laurea in Ingegneria e Scienze Informatiche

**Generazione automatica di dataset
text-to-SPARQL per fine-tuning di
LLM su knowledge graph
clinico-oncologico**

Tesi di laurea in:
PROGRAMMAZIONE

Relatore

Prof.ssa Antonella Carbonaro

Candidato

Matteo Tonelli

Sommario

Indice

Sommario	iii
1 Introduzione	1
2 Background	3
3 Analisi	5
3.1 Obiettivi	5
3.2 Vincoli	6
3.3 Requisiti funzionali	7
3.4 Requisiti non funzionali	7
4 Progettazione	9
4.1 Analisi dei problemi	9
4.2 Design del generatore	10
4.2.1 Modello del dominio	10
4.2.2 Sistema dei parametri	13
4.2.3 Meccanismo di estensione	14
4.3 Design dell'augmentation	15
4.3.1 Mascheramento dei valori letterali	15
4.3.2 Generazione iterativa e validazione	16
4.3.3 Varietà stilistica	17
5 Implementazione	19
5.1 Implementazione del generatore	19
5.1.1 Architettura a mixin	19
5.1.2 Scoperta della mappa del grafo	22
5.1.3 Costruzione delle route	24
5.1.4 Pianificazione: stima combinatoria e campionamento	25
5.1.5 Esecuzione sequenziale e parallela	27
5.1.6 Costruzione dello scheletro: percorso online e offline	29

5.1.7	Riempimento dei valori	32
5.1.8	Generazione del prompt e tracciamento degli span	33
5.1.9	Controlli di qualità e deduplicazione	34
5.1.10	Generazione manuale	35
5.1.11	Configurazione dei tier da file esterno	37
5.1.12	Split del dataset	39
5.2	Implementazione dell'augmentation	40
5.2.1	Generazione dei placeholder e mascheramento	40
5.2.2	Riformulazione booleana e suggerimenti naturali	41
5.2.3	Suddivisione in chunk per record complessi	42
5.2.4	Generazione iterativa: stile, retry e fallback	43
5.2.5	Validazione: due funzioni per due modalità	44
5.2.6	Rotazione degli stili e memoria anti-ripetizione	45
5.2.7	Statistiche di qualità e resilienza	46
5.2.8	La modalità senza mascheramento	47
6	Valutazione	49
7	Conclusioni e sviluppi futuri	51

Elenco delle figure

4.1	Modello di dominio semplificato: Patient come nodo centrale, con una classe rappresentativa per ciascuno dei sei gruppi di priorità . .	12
4.2	Modello esemplificativo dei parametri filtrabili	13

Elenco dei listati

5.1	Composizione della classe principale del generatore tramite mixin. .	19
5.2	Controllo dell'interruzione durante la generazione.	20
5.3	Costanti di classe utilizzate dal generatore.	21
5.4	Struttura semplificata della mappa del grafo, ridotta a titolo esemplificativo a una sola proprietà e una sola relazione dell'entità Patient.	22
5.5	Visita in profondità della mappa del grafo per la costruzione delle route disponibili a partire da un'entità radice.	24
5.6	Campionamento casuale di un insieme di route indipendenti tra loro per un singolo record.	26
5.7	Scelta tra esecuzione sequenziale e parallela in base al numero di worker configurati.	27
5.8	Configurazione condivisa usata nei run di produzione, per entrambe le liste di tier.	29
5.9	Lo scheletro appena ricostruito per i terminal gender e birth date , ancora privo di valori.	31
5.10	Riempimento di un nodo dello scheletro con valori concreti, distinguendo tra intervalli e insiemi discreti.	32
5.11	Lo stesso scheletro di listato 5.9, dopo il riempimento: elenco discreto per il campo categoriale, intervallo per quello di tipo data. .	32
5.12	Costruzione del prompt segmento per segmento, con tracciamento diretto della posizione di ciascun valore letterale.	33
5.13	Il primo controllo: verifica che i vincoli scelti siano effettivamente presenti nella query SPARQL restituita.	34
5.14	Il secondo controllo: il parsing sintattico tramite rdflib è condizionato dal parametro di configurazione.	35
5.15	Validazione di una configurazione di tier caricata da file esterno, prima di avviare la generazione.	38
5.16	Suddivisione del dataset in train, validation e test, stratificata per entità di origine.	39
5.17	Generazione dei placeholder e mascheramento del testo a partire dagli span di valore.	40

5.18	Eliminazione anticipata dei placeholder booleani, sostituiti con una forma proposizionale.	41
5.19	Suddivisione del testo mascherato in chunk, rispettando il vincolo sugli intervalli <i>between...and....</i>	42
5.20	Ciclo di generazione e validazione per un singolo chunk, con fallback esplicito in caso di fallimento persistente.	43
5.21	La funzione di validazione condivisa da chunk e varianti complete, con soglia di somiglianza adattiva.	44
5.22	Assegnazione ciclica degli stili e memoria a finestra delle generazioni recenti.	45
5.23	Calcolo della resa complessiva di un run, a partire dagli slot totali e da quelli completamente falliti.	46
5.24	La modalità senza mascheramento, mantenuta come termine di paragone esplicito e non come percorso di produzione alternativo. . . .	47

Capitolo 1

Introduzione

Structure of the Thesis

Capitolo 2

Background

Questo capitolo introduce le tecnologie e i concetti fondamentali necessari per la comprensione del contributo della tesi. Nelle sezioni successive viene presentato il Cancer Virtual Lab, il progetto nel cui ambito si colloca il lavoro descritto, seguito dai concetti di knowledge graph e SPARQL, dal problema del text-to-SPARQL e dalle sue difficoltà specifiche nel dominio clinico, dai lavori correlati sulla costruzione di dataset per questo compito e, infine, dai rischi specifici dell’augmentation testuale basata su modelli linguistici di grandi dimensioni.

Capitolo 3

Analisi

3.1 Obiettivi

L'obiettivo primario del progetto è la creazione di una pipeline in grado di costruire di un dataset annotato per il fine-tuning di un modello linguistico di dimensioni contenute, destinato alla traduzione di domande in linguaggio naturale in query SPARQL da eseguire sul knowledge graph del Cancer Virtual Lab. Da questo obiettivo generale derivano quattro obiettivi più specifici, che hanno guidato le scelte progettuali descritte nei capitoli successivi.

- O1** *Copertura selettiva ed estendibilità.* Il dataset deve coprire le parti dello schema del knowledge graph più rilevanti per l'applicazione, senza richiedere la copertura completa dello schema. Il sistema deve inoltre permettere di estendere progressivamente questa copertura aggiungendo nuove classi e relazioni, senza richiedere una riprogettazione del generatore.
- O2** *Varietà linguistica autentica.* Le domande generate devono presentare una varietà linguistica sufficiente a evitare formulazioni ripetitive o riconducibili a un unico template, mantenendo invariato il significato espresso dalla query sottostante.
- O3** *Tracciabilità completa.* Ogni domanda generata deve essere associata in modo univoco alla query SPARQL da cui deriva. L'associazione tra doman-

da e query deve essere quindi conservata esplicitamente, così da rendere verificabile ogni coppia utilizzata nel dataset.

- O4** *Volume di dati adeguato al fine-tuning.* Il dataset deve contenere un numero di esempi sufficiente per supportare il fine-tuning del modello linguistico considerato e deve garantire una copertura adeguata delle diverse tipologie di query previste.

3.2 Vincoli

Il progetto è stato condotto sotto un insieme di vincoli tecnici che ne hanno influenzato le scelte architettureali, descritte nel dettaglio nei capitoli successivi.

- V1** *Compatibilità con uno schema esistente e non modificabile.* Il sistema deve generare query conformi allo schema dell'ontologia del Cancer Virtual Lab, preesistente al progetto e non alterabile nel suo ambito. Nomi di classi, proprietà e relazioni sono vincoli esterni, non negoziabili in funzione di una generazione più semplice o di una copertura più ampia.
- V2** *Trattamento di dati clinici sensibili.* I valori coinvolti nelle query come identificativi di pazienti, codici di classificazione clinica o date sono dati clinici il cui significato deve rimanere verificabile e inalterato in ogni fase del processo. Questo vincolo esclude, inoltre, l'invio di tali dati a servizi esterni non controllati, imponendo l'esecuzione locale di qualunque componente del sistema che vi acceda.
- V3** *Capacità limitata del modello linguistico impiegato.* A causa del vincolo sull'esecuzione locale imposto da V2, il sistema potrà sfruttare solo modelli di dimensioni molto contenute, non potendo quindi fare affidamento sulla correttezza semantica spontanea dell'output prodotto, e richiede un meccanismo di validazione esplicito e indipendente dal modello stesso.

3.3 Requisiti funzionali

I requisiti funzionali descrivono cosa il sistema deve produrre, indipendentemente da come tale produzione venga realizzata.

- RF1** *Generazione di coppie prompt-query.* Il sistema deve generare coppie costituite da una query SPARQL sintatticamente valida e da un prompt in linguaggio naturale corrispondente, a partire dallo schema del knowledge graph.
- RF2** *Produzione di varianti multiple.* Il sistema deve produrre, per ciascuna coppia generata, un numero variabile di varianti parafrasate del medesimo prompt, preservando invariata la query associata.
- RF3** *Preservazione dei valori letterali.* Il sistema deve garantire, in ciascuna variante parafrasata, la presenza dei valori letterali presenti nella query, quali identificativi di pazienti, codici di classificazione clinica o date, nella loro forma esatta o in una forma alternativa verificabile.
- RF4** *Complessità variabile delle query.* Il sistema deve supportare la generazione di query di complessità variabile, dal caso di un singolo vincolo su una singola proprietà fino a query che attraversano più relazioni e combinano più vincoli sullo stesso nodo.

3.4 Requisiti non funzionali

I requisiti non funzionali riguardano invece proprietà trasversali del sistema, non direttamente osservabili nel singolo output prodotto.

- RNF1** *Sostenibilità computazionale.* Il sistema deve garantire un tasso di accettazione delle generazioni sufficientemente elevato da rendere il processo economicamente sostenibile in termini di tempo di calcolo.
- RNF2** *Riproducibilità in caso di interruzione.* Il sistema deve essere in grado di riprendere una generazione interrotta senza ripetere il lavoro già completato.

RNF3 *Robustezza semantica.* Il sistema deve limitare il rischio che la variazione linguistica modifichi, senza rilevarlo, il significato della domanda rispetto alla query sottostante, con particolare attenzione ai valori clinici controllati.

Nota *La robustezza semantica riguarda quindi la fedeltà del contenuto, mentre la tracciabilità di O3 riguarda la provenienza e l'associazione tra testo e query.*

Capitolo 4

Progettazione

4.1 Analisi dei problemi

I requisiti e i vincoli descritti nel capitolo precedente si traducono in tre sotto-problemi progettuali distinti, la cui interazione determina la difficoltà complessiva del compito. La distinzione è utile perché consente di separare le difficoltà legate alla costruzione delle query da quelle introdotte dalla variazione linguistica e, infine, dalla necessità di verificare gli output prodotti.

P1 *Sotto-problema combinatorio.* Dato lo schema del knowledge graph, generare query SPARQL sintatticamente corrette che coprano una distribuzione rappresentativa dell'uso reale del sistema, senza produrre un numero di combinazioni non gestibile nei tempi o eccessivo per un corretto fine-tuning.

P2 *Sotto-problema linguistico.* Dato un prompt in forma template privo di variazione, generare un numero variabile di parafrasi che siano al tempo stesso fluenti in linguaggio naturale e fedeli al contenuto semantico della query sottostante. A differenza di P1, qui la difficoltà non risiede nella dimensione dello spazio delle soluzioni possibili, ma nell'assenza di un criterio puramente formale per distinguere una parafrasi corretta da una scorretta. Una parafrasi può infatti essere grammaticalmente ineccepibile e superare ogni controllo meccanico applicabile, e tuttavia alterare il significato della domanda originale.

P3 *Sotto-problema di validazione.* Distinto da P1 e P2 ma dipendente da entrambi: verificare, per ciascuna variante generata, che la relazione con la query associata sia corretta e che il contenuto semantico non sia stato alterato durante la variazione linguistica. La tracciabilità richiesta da O3 è una proprietà verificabile della provenienza del dato; la robustezza semantica di RNF3 introduce invece una difficoltà ulteriore, poiché non esiste un criterio puramente formale che stabilisca con certezza l'equivalenza tra una formulazione in linguaggio naturale e la rappresentazione SPARQL sottostante. P3 è quindi il punto in cui la pipeline deve combinare controlli strutturali e semantici, ed è discusso nel dettaglio nel capitolo di valutazione.

I tre sotto-problemi individuati nell'analisi vengono affrontati da componenti distinti della pipeline. P1 è affrontato attraverso il generatore parametrico di query, che consente di controllare lo spazio delle configurazioni esplorate. P2 è affrontato attraverso la fase di augmentation linguistica, progettata per produrre formulazioni diverse a partire dalla medesima query. P3 richiede invece una fase di validazione, che verifica gli output prodotti dalle fasi precedenti prima del loro inserimento nel dataset.

4.2 Design del generatore

Questa sezione descrive le decisioni progettuali alla base del primo stadio del sistema, responsabile della generazione strutturata delle coppie prompt-query.

4.2.1 Modello del dominio

Il modello di dominio di questo progetto non coincide con l'intero schema del knowledge graph del Cancer Virtual Lab, ma con un sottoinsieme di 45 classi fornito dall'IRST. La selezione non ha seguito un criterio puramente tecnico, quale l'inclusione di ogni classe disponibile nel triplestore, ma è stata guidata dagli obiettivi clinici definiti a monte del progetto. Questo sottoinsieme, e la struttura relazionale che lo lega, costituisce un dato di fatto imposto dal knowledge graph ricevuto, non una scelta progettuale: le decisioni discusse nel resto di questa sezione riguardano non quali classi includere, ma come organizzarle e pesarle ai fini della generazione.

Le quarantacinque classi sono organizzate in gruppi di priorità decrescente, come in figura 4.1, secondo un criterio di distanza strutturale dal nodo Patient. L'adozione di questo criterio risponde a un'osservazione sul dominio applicativo stesso: Patient costituisce la radice naturale del grafo clinico, coerentemente con il modo in cui un clinico formula una domanda, ovvero a partire dal paziente, non da una classe periferica quale può essere un singolo biomarcatore. La distanza dal nodo radice offre, in aggiunta, un criterio oggettivo e calcolabile direttamente dalla struttura del grafo, senza richiedere l'imposizione anticipata di un giudizio soggettivo su quale classe sia più rilevante dal punto di vista clinico. Tale distanza si è inoltre rivelata un proxy efficace per la complessità della query generabile: una classe raggiungibile con un solo passaggio relazionale produce tipicamente domande semplici e dirette, mentre una classe raggiungibile con due o tre passaggi richiede query che concatenano più relazioni, rendendo la distanza informativa non solo rispetto alla rilevanza clinica, ma anche rispetto alla difficoltà attesa della domanda-tipo associata a ciascuna classe.

L'assenza di un simile criterio di priorizzazione avrebbe comportato conseguenze dirette sul dataset prodotto, discusse in relazione a O4 nel capitolo di analisi. Trattando le quarantacinque classi come equivalenti, il sistema avrebbe allocato un budget di generazione identico a ciascuna, indipendentemente dalla sua centralità nel dominio clinico. Classi concettualmente centrali, quali Patient, Diagnosis e Therapy, avrebbero ricevuto una rappresentazione insufficiente rispetto alla frequenza con cui compaiono nell'uso reale, mentre classi periferiche e altamente granulari, quali le singole categorie della classificazione TNM, avrebbero prodotto un numero di esempi ridondante rispetto alla loro effettiva rilevanza. Il risultato atteso di un simile sbilanciamento non sarebbe stato un dataset semplicemente meno efficiente, ma un modello allenato su di esso incline a generalizzare peggio proprio sulle categorie di domande più frequenti nell'uso previsto del sistema.

La distanza dal nodo Patient non costituisce l'unico criterio di raggruppamento adottato nel corso del progetto. Un secondo raggruppamento, tematico anziché strutturale, organizza le medesime quarantacinque classi per area clinica (demo-grafia, stadiazione, trattamento, ecc.) ed è stato introdotto in una fase successiva del progetto, discussa nel capitolo di implementazione. I due criteri non sono stati concepiti come alternativi, ma come complementari; di fatti la distanza struttu-

rale fornisce una progressione di complessità pulita, adatta a un primo passaggio di generazione, mentre il raggruppamento tematico ricombina le stesse classi secondo relazioni di co-occorrenza diverse, consentendo a un secondo passaggio di generazione di produrre query multi-relazionali strutturalmente nuove, anziché ricampionare gli stessi pattern già coperti dal primo raggruppamento.

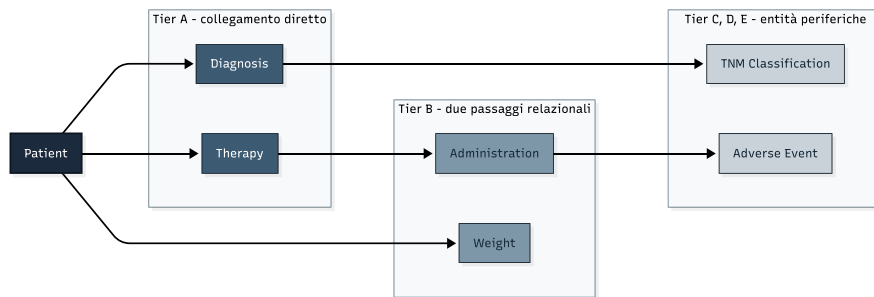


Figura 4.1: Modello di dominio semplificato: Patient come nodo centrale, con una classe rappresentativa per ciascuno dei sei gruppi di priorità

Il numero di record generati per ciascun gruppo decresce dal centro alla periferia del dominio ma tale decrescita non segue una progressione lineare rigida. Questo andamento è coerente con una calibrazione empirica, ovvero condotta bilanciando l'equilibrio finale del dataset, piuttosto che con l'applicazione di una formula matematica definita anticipatamente. Lo stesso principio di calibrazione empirica viene applicato al sistema dei quattro parametri descritto nella sezione seguente.

In sintesi, il dataset text-to-SPARQL prodotto da questo sistema non è generato in modo uniforme sull'intero schema disponibile, ma segue una gerarchia di rilevanza clinica propria del progetto. Le classi incluse e i parametri di complessità applicati a ciascuna sono stati scelti con l'obiettivo di riflettere le tipologie di domande ritenute più rilevanti nel contesto del Cancer Virtual Lab, privilegiando la rilevanza del dominio rispetto alla massimizzazione della copertura formale del grafo.

4.2.2 Sistema dei parametri

Il modello di dominio descritto nella sezione precedente definisce quali classi partecipano alla generazione e con quale peso relativo, ma non specifica in alcun modo la complessità delle singole query generabili su ciascuna di esse.

Il generatore espone quattro parametri indipendenti, ciascuno regolabile entro un intervallo minimo e massimo, la cui combinazione determina la complessità della query risultante:

- il numero di relazioni attraversate nel grafo a partire dal nodo radice;
- il numero di percorsi alternativi combinati nella medesima query tramite clausole di disgiunzione;
- il numero di proprietà vincolate su un medesimo nodo;
- il numero di valori ammessi per ciascun vincolo.

Questi quattro parametri corrispondono a dimensioni strutturalmente distinte lungo cui una query SPARQL può crescere in complessità. La loro separazione consente di variare un aspetto della complessità senza doverlo necessariamente far crescere insieme agli altri. Trattare la complessità come una singola grandezza scalare avrebbe nascosto questa distinzione, impedendo un controllo mirato su quale aspetto specifico della query si desidera far variare per una determinata classe o gruppo di classi.

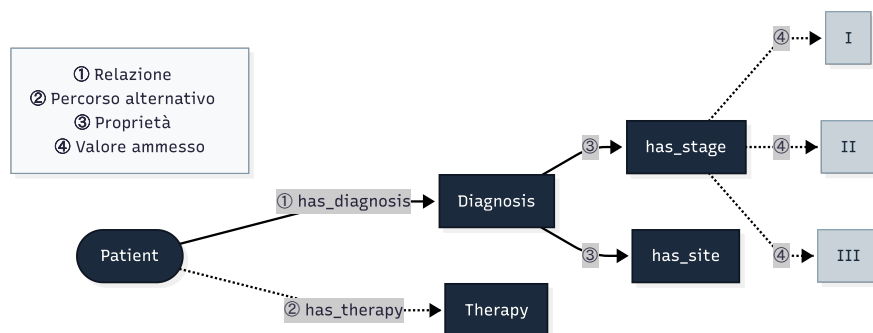


Figura 4.2: Modello esemplificativo dei parametri filtrabili

Sebbene l'impianto concettuale dei quattro parametri sia rimasto invariato nel corso del progetto, i valori numerici effettivamente assegnati a ciascuno di essi non sono stati definiti una sola volta, ma calibrati progressivamente attraverso diverse configurazioni successive, tutte vincolate da un limite superiore imposto dall'endpoint SPARQL stesso. Infatti query eccessivamente complesse, al crescere del numero di relazioni e proprietà vincolate simultaneamente, incorrevano in timeout o fallivano l'esecuzione contro il triplestore reale, rendendo necessario mantenere ciascun parametro entro un tetto massimo compatibile con le capacità dell'endpoint.

4.2.3 Meccanismo di estensione

Il sistema di parametri descritto nella sezione precedente offre, come conseguenza diretta della sua struttura, una capacità che va oltre la sola configurazione iniziale della complessità: la possibilità di ampliare il dataset generato senza rigenerare le porzioni già prodotte, e senza il rischio che le nuove query si sovrappongano a quelle esistenti.

Questa capacità è resa possibile dal fatto che un intervallo minimo-massimo definisce uno spazio di configurazioni delimitato con precisione. Impostando, per un nuovo preset di generazione, un intervallo disgiunto da quello di un preset precedente su almeno una dimensione rilevante, i due spazi risultano disgiunti a livello di configurazione dei parametri. In questo modo il nuovo preset esplora combinazioni che non appartenevano allo spazio configurato in precedenza. La proprietà è quindi garantita a livello parametrico; l'assenza di duplicati a livello di query è invece una proprietà che dipende dal modo in cui le configurazioni vengono trasformate in query e, se necessario, può essere verificata esplicitamente durante la generazione.

Questa distinzione rende il meccanismo di estensione più preciso: l'aggiunta di nuove porzioni al dataset non richiede una riprogettazione del generatore, ma la definizione di nuovi preset i cui intervalli siano scelti in relazione a quelli già utilizzati. La procedura può essere applicata ripetutamente, mantenendo traccia dello spazio parametrico già esplorato e introducendo nuove configurazioni al di fuori di esso.

4.3 Design dell'augmentation

Questa sezione descrive le decisioni progettuali alla base del secondo stadio del sistema, responsabile della trasformazione delle coppie prompt-query prodotte dal generatore in più varianti linguistiche naturali, secondo il compromesso tra varietà e fedeltà semantica richiesto da O2 e la robustezza semantica richiesta da RNF3.

4.3.1 Mascheramento dei valori letterali

Il problema affrontato da questo stadio è duplice: da un lato produrre varie riformulazioni linguisticamente a partire da uno stesso prompt, dall'altro garantire che tale variazione non alteri i valori letterali che compaiono nel prompt e che sono referenziati dalla query SPARQL associata (identificativi, codici di classificazione clinica, date). Lasciare che un modello linguistico riscriva liberamente l'intero testo, valori inclusi, esporrebbe il sistema al rischio già discusso in RF3 e RNF3, ovvero una riformulazione apparentemente valida ma silenziosamente scorretta rispetto alla query originale.

La soluzione adottata separa la variazione linguistica dai valori da preservare tramite un meccanismo di mascheramento. Prima di essere sottoposto al modello, ogni valore letterale identificato nel prompt viene sostituito da un token segnaposto opaco; il modello riformula quindi il testo mascherato, e solo al termine della generazione i segnaposto vengono sostituiti nuovamente con i valori originali. In questo modo il modello non ha mai accesso al valore letterale reale durante la riformulazione, e può quindi soltanto ricollocarlo all'interno di una struttura sintattica diversa.

Sono stati introdotti inoltre due accorgimenti specifici per la gestione dei valori mascherati. Il primo riguarda quando un segnaposto rappresenta una condizione vera o falsa, come la presenza di un evento avverso; in questo caso il valore non viene mantenuto nella forma letterale *true* o *false*, ma viene convertito in una formulazione più naturale per il linguaggio umano, come «con» o «senza», seguita dal termine clinico corrispondente.

Il secondo accorgimento riguarda i record caratterizzati da un numero elevato di valori mascherati. In questi casi, il testo non viene sottoposto al modello attraverso

un'unica chiamata, ma viene suddiviso in più porzioni, ciascuna contenente al massimo un numero prefissato di segnaposto. Le singole porzioni vengono quindi riformulate separatamente e ricomposte al termine della generazione. Questa scelta deriva da un'osservazione empirica secondo cui, oltre una determinata soglia di segnaposto presenti nello stesso prompt, la qualità delle riformulazioni generate tende a diminuire.

4.3.2 Generazione iterativa e validazione

Il mascheramento garantisce che i valori letterali, se presenti nell'output, siano corretti; non garantisce però che il modello produca sempre un output valido al primo tentativo, né che varianti successive dello stesso record non finiscano per ripetersi tra loro, vanificando l'obiettivo di varietà linguistica di O2.

Il sistema affronta questo problema attraverso un ciclo iterativo di generazione e verifica. Ogni candidato prodotto dal modello viene sottoposto a una fase di validazione che considera diversi criteri.

- **Preservazione dei segnaposto.** Il candidato viene rifiutato se anche un solo segnaposto è stato omesso, modificato o sostituito.
- **Assenza di segnaposto non previsti.** Viene verificato che il testo non contenga sequenze che imitano la sintassi di un segnaposto senza corrispondere a uno dei segnaposto effettivamente presenti nel testo originale.
- **Controllo della similarità.** Il candidato viene confrontato con le varianti già accettate per lo stesso record. Se risulta eccessivamente simile a una di esse, viene rifiutato. La soglia di similarità viene resa progressivamente più permissiva al diminuire della lunghezza del testo, così da evitare che formulazioni brevi vengano considerate erroneamente come duplicati.

In caso di rifiuto, il sistema non si limita a ripetere la generazione. Il candidato scartato e il relativo motivo vengono forniti al modello come contesto aggiuntivo per il tentativo successivo, in modo da ridurre la probabilità che venga ripetuto lo stesso errore. A ogni nuovo tentativo viene inoltre incrementata progressivamente

la temperatura di campionamento, favorendo la produzione di una formulazione diversa da quella precedente.

Se anche l'ultimo tentativo disponibile viene rifiutato, il sistema non forza comunque l'inserimento di una nuova variante. Viene invece mantenuto il testo originale non modificato e viene registrato esplicitamente il ricorso a questo comportamento di ripiego. In questo modo si evita di accettare silenziosamente una formulazione che violi i vincoli definiti da RF3 o RNF3.

Il numero di varianti richieste per ciascun record non è inoltre fissato in modo uniforme, ma proporzionato al numero di valori mascherati nel record: un prompt con pochi valori offre in pratica poco margine di riformulazione prima di esaurire le combinazioni naturali, per cui imporre lo stesso numero di varianti richieste a un record semplice e a uno complesso avrebbe prodotto, nel primo caso, un'alta proporzione di tentativi rifiutati per eccessiva similarità.

4.3.3 Varietà stilistica

Il solo mascheramento non garantisce, di per sé, la varietà linguistica richiesta da O2: un modello lasciato libero di riformulare tende a convergere su un piccolo numero di strutture sintattiche ricorrenti. Il sistema affronta questo problema definendo un insieme di dieci registri stilistici predefiniti (ad esempio una formulazione tecnica in stile interrogazione a database, una clinico-formale, una colloquiale, una statistica orientata al conteggio), esplicitamente descritti al modello a ogni chiamata.

Gli stili non vengono assegnati alle varianti in modo casuale, ma percorsi in sequenza ciclica, in modo che la distribuzione degli stili sulle varianti generate risulti uniforme anziché soggetta alla variabilità di un'estrazione casuale. Per ciascuno stile, il sistema mantiene inoltre una memoria a breve termine delle formulazioni e delle parole di apertura usate più di recente: un nuovo candidato che ripropone un'apertura già usata di recente per lo stesso stile viene rifiutato dalla validazione allo stesso modo di una riformulazione troppo simile a livello di intero testo, estendendo così il controllo di varietà anche alla struttura delle prime parole della frase, non solo al suo contenuto complessivo.

Il sistema conserva infine, per ciascun record, anche una modalità alternativa priva di mascheramento, che riproduce il comportamento originario a chiamata singola sull'intero testo. Questa modalità non costituisce un secondo percorso di produzione, ma un termine di paragone esplicito, i cui risultati sono discussi assieme a quelli della modalità con mascheramento nel capitolo di valutazione.

Capitolo 5

Implementazione

In questo capitolo viene descritto come sono state implementate le varie parti del contributo di questa tesi, in maniera coerente con la progettazione svolta nel capitolo 4. La trattazione segue l'ordine cronologico in cui i due stadi del sistema sono stati sviluppati.

5.1 Implementazione del generatore

5.1.1 Architettura a mixin

Il generatore è organizzato attorno a un'unica classe, `BulkDatasetGenerator`, ottenuta mediante ereditarietà multipla da sei *mixin*¹. Questa struttura permette di suddividere l'implementazione in responsabilità distinte, mantenendo al tempo stesso un'unica interfaccia per l'intero processo di generazione. La scelta è coerente con l'obiettivo di mantenere il generatore estendibile: le diverse fasi della pipeline possono infatti essere modificate o ampliate intervenendo principalmente sul componente che ne implementa la relativa responsabilità. La composizione della classe principale è riportata nel listato 5.1.

Listing 5.1: Composizione della classe principale del generatore tramite mixin.

```
1 class BulkDatasetGenerator(  
2     FilterApiMixin,
```

¹Un mixin è una classe pensata per essere combinata con altre tramite ereditarietà multipla, fornendo un insieme coeso di funzionalità senza costituire di per sé una gerarchia di tipi completa.

```

3      DiscoveryMixin,
4      SkeletonMixin,
5      PlanningMixin,
6      ProgressMixin,
7      OutputMixin,
8  ):
9      # parameters

```

In particolare le divisioni consistono in:

- **FilterApiMixin** incapsula le chiamate di rete verso l'API interna del CVL;
- **DiscoveryMixin** costruisce la mappa del grafo dello schema;
- **SkeletonMixin** costruisce lo scheletro di una singola query;
- **PlanningMixin** determina quali combinazioni di proprietà devono essere campionate;
- **OutputMixin** esegue i controlli di qualità finali e gestisce la scrittura dei risultati su disco;
- **ProgressMixin** gestisce gli aspetti legati all'interazione con l'interfaccia Streamlit, tra cui l'aggiornamento della barra di avanzamento e la possibilità di interrompere l'esecuzione.

Le responsabilità non sono quindi indipendenti, ma cooperano secondo il flusso della generazione, tutte fatta ad eccezione di **ProgressMixin** che costituisce invece una responsabilità trasversale, poiché può intervenire durante l'esecuzione delle diverse fasi senza rappresentare una fase autonoma della pipeline.

La separazione è particolarmente evidente nella gestione dell'interruzione della generazione (gestita proprio da **ProgressMixin**). Il metodo `_is_aborted()`, riportato nel listato 5.2, viene richiamato periodicamente durante la generazione per verificare se l'esecuzione debba essere interrotta.

Listing 5.2: Controllo dell'interruzione durante la generazione.

```

1  def _is_aborted(self) -> bool:
2      if self._abort_event.is_set():
3          return True
4      if not self.config.get("abortable", True):

```

```
5         return False
6     if self._can_paint() and bool(
7         st.session_state.get("abort_generation", False)
8     ):
9         self._abort_event.set()
10    return self._abort_event.is_set()
```

Il controllo combina due meccanismi. Il primo è l'evento di sincronizzazione interno `_abort_event`, che rappresenta lo stato di interruzione del run. Il secondo è lo stato dell'interfaccia Streamlit, dal quale viene letto il valore di `session_state["abort_generation"]`. Quando l'utente richiede l'interruzione, il metodo imposta l'evento interno; le chiamate successive possono quindi rilevare direttamente tale stato senza dover interrogare nuovamente l'interfaccia.

Il controllo dello stato di Streamlit viene eseguito soltanto quando l'esecuzione è configurata come interrompibile, tramite `config["abortable"]`, e quando il contesto consente di interagire con l'interfaccia. In questo modo la stessa implementazione può essere utilizzata anche nei contesti che non espongono un controllo di interruzione, come le esecuzioni da riga di comando o un pannello di generazione batch.

Le costanti di classe di `BulkDatasetGenerator`, riportate nel listato 5.3, definiscono alcuni parametri di funzionamento condivisi dalle diverse fasi della generazione.

Listing 5.3: Costanti di classe utilizzate dal generatore.

```
1 class BulkDatasetGenerator(
2     # parameters
3 ):
4     BUFFER_SIZE = 50
5     MAX_DISCOVERY_WORKERS = 8
6     GENERATION_WORKERS = 4
7     DISCARD_SAMPLES = 500
8     VALUE_REROLLS = 4
9     MAX_STORED_VALUES = 5000
10    SAMPLING_GIVEUP_STREAK = 25
```

In particolare, tali parametri controllano la dimensione del buffer di scrittura, il grado di parallelismo utilizzato durante la scoperta e la generazione, il numero massimo di valori conservati per proprietà e la soglia di tentativi consecutivi senza successo oltre la quale il campionamento viene interrotto. A questi si aggiungono

i parametri relativi alla resilienza delle chiamate di rete, costituiti da un timeout di base di 15 secondi, fino a due ripetizioni in caso di fallimento e un secondo di attesa tra un tentativo e l'altro.

5.1.2 Scoperta della mappa del grafo

Prima di generare qualunque record, il sistema costruisce un'unica mappa completa delle proprietà e delle relazioni disponibili per ciascuna entità dello schema, interrogando l'API interna della piattaforma CVL. Questa mappa viene costruita una sola volta e riutilizzata per l'intera esecuzione, evitando di ripetere le stesse chiamate di rete ogni volta che un'entità già scoperta viene esplorata di nuovo. Il dettaglio implementativo di come l'API venga interrogata non è discusso in questa sede, trattandosi di un'interfaccia della piattaforma CVL e non di un contributo originale di questa tesi; qui rileva solo il risultato che il generatore ne ricava, riportato in forma semplificata nel listato 5.4: per ogni entità, l'insieme delle sue proprietà filtrabili (**properties**) e delle entità verso cui può proseguire una relazione (**relations**).

Listing 5.4: Struttura semplificata della mappa del grafo, ridotta a titolo esemplificativo a una sola proprietà e una sola relazione dell'entità Patient.

```
1 graph_map = {
2   "properties": {
3     "Patient": [
4       {
5         "predicate": {"value": "has_gender", "label": "gender"},
6         "datatype": "string",
7         "n_values": 2,
8         "values": [
9           {"value": "F", "label": "Female"},
10          {"value": "M", "label": "Male"},
11        ],
12      },
13    ],
14  },
15  "relations": {
16    "Patient": [
17      {"predicate": {"value": "has_diagnosis", "label": "diagnosis"},
18       "target": {"value": "Diagnosis", "label": "Diagnosis"}},
19    ],
20  },
21 }
```


Ogni proprietà filtrabile porta con sé, oltre al predicato che la identifica, il tipo di dato, il numero di valori distinti disponibili (`n_values`) e i valori stessi, già raccolti in questa fase (fino al tetto `MAX_STORED_VALUES` descritto più sotto); ogni relazione porta invece il predicato usato per il salto e l'entità di destinazione (`target`).

La scelta di indicizzare entrambe le sezioni per nome di entità, invece di usare una singola lista di record, non è casuale. La mappa viene infatti interrogata ripetutamente durante la scoperta delle route (sezione 5.1.3), una volta per ciascuna entità di volta in volta esplorata, e un dizionario permette di recuperare le proprietà o le relazioni di una data entità in tempo costante, invece di dover scorrere l'intera struttura ogni volta.

Costruire la mappa una volta sola, prima di iniziare qualunque generazione, comporta anche un compromesso più ampio. Quando viene aperta, ogni entità viene interrogata come punto di partenza isolato, con una storia di selezione vuota, indipendentemente dal fatto che nell'esplorazione vera delle route quella stessa entità venga poi raggiunta attraverso un percorso diretto o attraverso più salti relazionali. La mappa assume quindi che le proprietà e le relazioni disponibili per un'entità dipendano solo dal suo tipo, non dal percorso concreto con cui la si raggiunge, a differenza dell'API di CVL, che può restituire relazioni diverse a seconda della storia di selezione accumulata fino a quel punto. Questo costo viene accettato deliberatamente, perché l'alternativa, interrogare l'API da capo ogni volta che una qualunque entità viene esplorata, è proprio il collo di bottiglia che si vuole evitare.

Per ridurre il tempo complessivo di questa fase, le interrogazioni relative alle diverse opzioni di una stessa entità vengono eseguite in parallelo, fino a un massimo configurabile per volta (`MAX_DISCOVERY_WORKERS`), tramite un pool di thread; i valori restituiti per ciascuna proprietà filtrabile vengono conservati fino a un tetto massimo (`MAX_STORED_VALUES`), per non tenere in memoria un elenco potenzialmente enorme quando un campo ammette moltissimi valori distinti.

5.1.3 Costruzione delle route

Con la mappa disponibile, per ciascuna entità radice richiesta, si esplora tramite una visita in profondità (DFS) ogni combinazione raggiungibile di salti relazionali fino a un limite di profondità configurato.

Listing 5.5: Visita in profondità della mappa del grafo per la costruzione delle route disponibili a partire da un'entità radice.

```
1 def dfs(type_value, chain, relations_used, visited, ancestor_terminals):
2     terminals = [
3         {"predicate": p["predicate"], "chain_prefix": list(chain), ...}
4         for p in properties.get(type_value, [])
5     ]
6     if terminals:
7         routes.append({
8             "chain": list(chain),
9             "terminals": ancestor_terminals
10            + [{"t", "is_final_entity": True} for t in terminals],
11        })
12
13     if relations_used >= max_relations:
14         return
15     inherited = ancestor_terminals + [{"t", "is_final_entity": False} for t in
16                                     terminals]
17     for rel in relations.get(type_value, []):
18         target_value = (rel.get("target") or {}).get("value")
19         if not target_value or target_value in visited:
20             continue
21         dfs(
22             target_value,
23             [*chain, {"predicate": rel["predicate"], "reference_to": rel.get("
24                 target")}],
25             relations_used + 1, visited | {target_value}, inherited,
```

Ogni volta che la visita raggiunge un nodo con proprietà filtrabili, viene registrata una *route*, ovvero l'insieme di:

- la sequenza di salti relazionali per raggiungerlo (la *chain*);
- l'insieme delle proprietà disponibili in quel punto (i *terminal*), che include sia quelle del nodo corrente sia quelle ereditate dai nodi attraversati in precedenza.

La distinzione dei terminal in proprietà ereditate e non (tramite variabile booleana) garantisce, nel passo successivo di campionamento, che ogni combinazione di proprietà scelta includa almeno un vincolo sul nodo in cui la route si ferma davvero, evitando di costruire query che vincolano solo nodi di passaggio.

Sebbene la visita che le genera sia intrinsecamente ad albero, l'insieme delle route risultanti viene appiattito in una singola lista di record indipendenti per facilitare la fase di campionamento (sezione 5.1.4), la quale deve poter scorrere ed estrarre a caso qualunque route tra tutte quelle disponibili, con accesso diretto e costante a ciascuna, senza dover ripercorrere un albero ogni volta per ricostruire chain e terminal di un nodo scelto casualmente al suo interno.

5.1.4 Pianificazione: stima combinatoria e campionamento

Prima di generare i record, il sistema calcola quante coppie prompt-query distinte siano effettivamente raggiungibili con le route disponibili e i vincoli configurati, in modo da non richiedere più record di quanti lo spazio combinatorio possa realmente offrire; solo dopo, campiona a caso tra le combinazioni così stimate per costruire ogni singolo record. I due passi sono di natura diversa: la stima è un conteggio puramente matematico, espresso di seguito con delle formule; il campionamento è invece un algoritmo vero e proprio, mostrato più avanti in codice.

Stima combinatoria

Sia r una route, T_r l'insieme dei suoi terminal e $T_r^f \subseteq T_r$ i terminal finali (quelli del nodo dove la route si ferma davvero). Sia $v(t)$ il numero di combinazioni di valori di un terminal, e $\mathcal{C}_k(T_r)$ le combinazioni di k terminal di T_r (nel senso standard, $\binom{|T_r|}{k}$ possibilità) che includono almeno un terminal di T_r^f . Con $p_{\min}$, $p_{\max}$ i limiti *min_properties*/*max_properties* configurati, il peso di r è

$$w(r) = \begin{cases} 0 & \text{se } T_r^f = \emptyset \\ \sum_{k=p_{\min}}^{\min(p_{\max}, |T_r|)} \sum_{c \in \mathcal{C}_k(T_r)} \prod_{t \in c} v(t) & \text{altrimenti.} \end{cases} \quad (5.1)$$

Un record spesso combina più route insieme. Sia V l'insieme delle route con peso non nullo, e $\text{indip}(S)$ vera quando nessuna coppia di route in S ha una chain prefisso dell'altra (sezione 5.1.3). Con $s_{\min}$, $s_{\max}$ i limiti *min_paths*/*max_paths* configurati, il tetto teorico dell'entità è

$$N = \sum_{s=s_{\min}}^{s_{\max}} \sum_{\substack{S \subseteq V \\ |S|=s \\ \text{indip}(S)}} \prod_{r \in S} w(r). \quad (5.2)$$

È da sottolineare che N è di fatto una sovrastima, in quanto route indipendenti che condividono un prefisso condividono anche parte dei terminal, mentre il prodotto dei pesi utilizzato nella nostra formula le tratta come scelte separate. Per l'uso che se ne fa, cioè riconoscere quando lo spazio delle combinazioni è troppo piccolo rispetto ai record richiesti, un limite superiore basta, perciò possiamo evitare di complicare ulteriormente il calcolo. Proprio per evitare un'ulteriore inefficienza, l'algoritmo imposta una soglia di 500 000 combinazioni da esaminare, dopo la quale il conteggio esatto viene abbandonato e il campionamento procede senza un tetto noto in anticipo.

Campionamento

Per ogni record da generare, il numero di route da combinare viene scelto a caso entro i limiti configurati. Le route disponibili vengono poi estratte da un elenco mescolato in ordine casuale, scartando quelle che risultano *in conflitto* con una già scelta, cioè la cui chain è un prefisso di un'altra già selezionata, il che indicherebbe due vincoli ridondanti sullo stesso tratto di percorso. Il campionamento continua finché non si raggiunge il numero di record stimato dal tetto teorico (equazione (5.2)), oppure finché 250 tentativi consecutivi (`SAMPLING_GIVEUP_STREAK`) non producono nulla di nuovo, segno che lo spazio raggiungibile è stato esaurito.

Listing 5.6: Campionamento casuale di un insieme di route indipendenti tra loro per un singolo record.

```

1 def sample_independent_combo(target_p):
2     pool = list(valid)
3     rng.shuffle(pool)
4     chosen, chosen_keys = [], []

```

```
5     for r in pool:
6         if len(chosen) >= target_p:
7             break
8         key = chain_key_of[id(r)]
9         if not routes_mod.conflicts(chosen_keys, key):
10             chosen.append(r)
11             chosen_keys.append(key)
12     return tuple(chosen)
```

Questa modalità di campionamento casuale (`_run_sampled`) non è l'unica prevista dal codice, esiste infatti anche una modalità a forza bruta (`_run_brute_force`) che enumera esaustivamente ogni combinazione possibile di route e proprietà. Quest'ultima modalità risulta utile quando l'obiettivo è ottenere una copertura completa dello spazio combinatorio; tuttavia non è stata utilizzata nei run di produzione, ma è stata comunque impiegata in fase di sviluppo per individuare eventuali errori di generazione, proprio grazie alla sua capacità di esplorare lo spazio delle combinazioni senza lasciarne fuori nessuna. Nella casistica affrontata da questa tesi però il dataset generato è destinato al fine-tuning, per il quale non è necessario enumerare tutte le combinazioni possibili, anzi è preferibile costruire un insieme sufficientemente ampio e vario di esempi rappresentativi, evitando al contempo di produrre una quantità di dati eccessiva rispetto allo scopo.

5.1.5 Esecuzione sequenziale e parallela

Una volta pianificata una combinazione di route e proprietà da trasformare in un record, ha inizio l'intero processo di costruzione del record vero e proprio, il quale comporta diverse chiamate di rete verso l'API del CVL. Questo processo può essere eseguito in sequenza, una combinazione alla volta, oppure in parallelo su più thread, ciascuno impegnato a costruire un record diverso in contemporanea agli altri, a seconda della configurazione.

Listing 5.7: Scelta tra esecuzione sequenziale e parallela in base al numero di worker configurati.

```
1 workers = max(1, int(self.config.get("num_workers") or self.GENERATION_WORKERS))
2 if workers <= 1:
3     self._run_jobs_sequential(jobs_iter, total_pairs, p, target, max_no_new)
4     return
```

Se il numero di worker configurato è maggiore di uno, le combinazioni campionate vengono sottomesse a un pool di thread, fino a `GENERATION_WORKERS` worker attivi contemporaneamente (di default 4), ciascuno impegnato nella costruzione di un record indipendente. Il sistema non sottomette tuttavia tutte le combinazioni disponibili in un'unica operazione, poiché ciò potrebbe determinare l'accumulo in coda di un numero molto elevato di job in attesa. Viene invece mantenuta una finestra di lavori sottomessi ma non ancora completati, di dimensione pari al doppio del numero di worker attivi; quando la generazione di un record termina e il risultato viene salvato, il relativo spazio viene immediatamente riutilizzato per sottomettere una nuova combinazione.

Questa strategia consente, in condizioni di parallelismo effettivo, di mantenere i worker occupati riducendo i tempi morti, senza introdurre una quantità di lavoro in coda eccessiva rispetto alla capacità di elaborazione disponibile. Il meccanismo è stato predisposto principalmente in vista di future esecuzioni su larga scala, nelle quali il parallelismo potrebbe ridurre significativamente il tempo complessivo di generazione.

Durante la produzione del dataset trattato, il numero di worker è stato invece impostato esplicitamente a 1 (listato 5.8) eseguendo così la generazione in modalità single-threaded. La scelta è dettata dal collo di bottiglia rappresentato dalle chiamate all'API, le cui richieste concorrenti non avrebbero ridotto i tempi in proporzione, anzi avrebbero rischiato di comprometterne stabilità e prestazioni. Il parallelismo è rimasto invece attivo nella scoperta della mappa (sezione 5.1.2), dove il carico delle operazioni si presta meglio all'esecuzione concorrente.

Proprio perché il parallelismo può comunque essere attivato lo stato condiviso tra i thread non è protetto da un unico lock globale ma da quattro lock distinti, ciascuno a guardia di una porzione di stato diversa e indipendente dalle altre: un lock protegge gli insiemi usati per riconoscere i duplicati (sezione 5.1.9), un altro la cache dello scheletro (sezione 5.1.6), un altro ancora le statistiche interne sulle chiamate effettuate, e l'ultimo il registro dei job attualmente in esecuzione. Tenerli separati, invece di usarne uno solo per tutto, evita che un thread resti bloccato in attesa di una struttura dati che non gli serve, solo perché un altro thread ne sta aggiornando una diversa nello stesso istante.

5.1.6 Costruzione dello scheletro: percorso online e offline

Ogni combinazione campionata deve essere tradotta in una struttura dati intermedia, chiamata *scheletro*, che rappresenta i nodi del grafo coinvolti nella query, prima ancora che i valori concreti vengano scelti. Il sistema offre due percorsi alternativi per costruirla, insieme ad altre opzioni di comportamento, tutte raccolte in un'unica configurazione condivisa da entrambe le liste di tier.

Le opzioni condivise

Listing 5.8: Configurazione condivisa usata nei run di produzione, per entrambe le liste di tier.

```
1 COMMON_CONFIG = {  
2     "num_workers": 1,  
3     "offline_history": True,  
4     "include_value_spans": True,  
5     "skip_navigation_in_prompt": False,  
6     "validate_sparql": True,  
7 }
```

Ciascuna di queste cinque opzioni governa un aspetto distinto del comportamento del generatore.

- **num_workers** a 1 è la scelta, già motivata in sezione 5.1.5, di eseguire la generazione in sequenza anziché in parallelo.
- **include_value_spans** controlla se il generatore debba tracciare le posizioni dei valori letterali nel prompt (sezione 5.1.8, dove è ripreso in dettaglio).
- **skip_navigation_in_prompt**, se attivato ometterebbe dal testo del prompt le clausole di sola navigazione, rendendolo più corto ma meno esplicito sul percorso nel grafo; è stato tenuto disattivato in produzione per non perdere quell'informazione.
- **validate_sparql** controlla se la query SPARQL restituita dall'API debba passare un controllo di sintassi prima di essere accettata; sarà ripreso più avanti insieme agli altri controlli di qualità (sezione 5.1.9).

- `offline_history` decide se lo scheletro venga ricostruito in locale oppure costruito camminando dal vivo dentro l'API; riguarda direttamente le due sottosezioni seguenti.

Il percorso online

Quando `offline_history` è disattivato, lo scheletro viene costruito camminando passo per passo tramite chiamate ricorsive nell'API del CVL, esplorando in modo consequenziale proprietà e relazioni. Per ogni terminal della combinazione, il sistema percorre uno per uno i salti della sua chain (sezione 5.1.3) e, a partire dall'entità radice, a ogni salto chiede all'API di proseguire dal nodo raggiunto fino a quel momento verso il nodo successivo, accumulando progressivamente lo stato della query in costruzione finché non raggiunge il nodo del terminal stesso. Questa procedura ripercorre esattamente il percorso che la fase di scoperta della mappa aveva già seguito per costruire la mappa iniziale (sezione 5.1.2); la differenza è che lì veniva fatto una volta per ogni proprietà isolata, per costruire la mappa, mentre qui viene ripetuto per intero a ogni singolo record. Durante questa fase di costruzione online sono inoltre presenti alcuni meccanismi di resilienza, introdotti per gestire le condizioni anomale e gli eventuali problemi che possono verificarsi durante le chiamate all'API. Tali meccanismi si sono rivelati necessari nel corso dello sviluppo e vengono descritti di seguito.

- se l'API impiega troppo a rispondere, il fallimento è considerato transitorio, perché un tentativo successivo potrebbe ancora avere successo, e non viene memorizzato in nessuna cache;
- se invece risponde ma senza risultati utili, il terminal è considerato irraggiungibile in modo permanente, con due conseguenze distinte:
 - viene memorizzato in una cache condivisa così che i tentativi successivi lo scartino subito senza ripetere l'esplorazione;
 - il record in corso esclude dalla combinazione quel singolo terminal e la generazione prosegue con gli altri rimasti (esiste una modalità più rigida, mai usata in produzione, che interromperebbe l'intero tentativo al primo terminal irraggiungibile);

- indipendentemente dai primi due, per non ripetere l'intera esplorazione quando la stessa combinazione di terminal ricorre in record diversi, lo scheletro completo viene tenuto in una cache separata, indicizzata sulla combinazione esatta richiesta, e riusato direttamente se si ripresenta, con solo il riempimento dei valori concreti (sezione 5.1.7) che cambia da un record all'altro.

Il percorso offline

Con `offline_history` attivo, come nei run di produzione, tutta la camminata appena descritta viene saltata e lo scheletro non si costruisce esplorando dal vivo l'API di CVL, ma viene ricostruito interamente in locale a partire dai dati che la scoperta della mappa aveva già raccolto per ciascuna proprietà (sezione 5.1.2). Per ogni terminal coinvolto nella combinazione, il sistema ricrea la stessa sequenza di nodi intermedi che un'esplorazione dal vivo avrebbe prodotto, camminando sulla sequenza di salti già nota (registrata al momento della scoperta delle route, sezione 5.1.3) e usando i valori possibili già memorizzati per quel terminal, senza doverli richiedere di nuovo; con lo scheletro pronto, si arriva così direttamente all'unico passaggio che nessuno dei due percorsi può evitare, ovvero la chiamata finale descritta di seguito.

Listing 5.9: Lo scheletro appena ricostruito per i terminal `gender` e `birth date`, ancora privo di valori.

```
1 history = [  
2     {  
3         "query_nodes": [  
4             {"predicate": {"label": "gender"}, "values": None, "value_range": None  
5             },  
6             {"predicate": {"label": "birth date"}, "values": None, "value_range":  
7                 None},  
8         ]  
9     }  
10 ]
```

Indipendentemente dal percorso scelto per lo scheletro, il testo effettivo della query SPARQL viene sempre ottenuto con un'unica chiamata finale all'API, che traduce la struttura accumulata in una query eseguibile. Questo passaggio non è mai eseguito localmente, in nessuna delle due modalità, rappresentando così l'unica

chiamata API fatta dal percorso offline. Su questa stessa chiamata finale interviene anche il quinto flag di listato 5.8, `validate_sparql`, ripreso e motivato più avanti insieme agli altri controlli di qualità applicati al risultato finale (sezione 5.1.9).

5.1.7 Riempimento dei valori

Una volta pronto lo scheletro, ogni suo nodo vuoto viene riempito con valori concreti scelti a caso dal pool associato al terminal corrispondente.

Listing 5.10: Riempimento di un nodo dello scheletro con valori concreti, distinguendo tra intervalli e insiemi discreti.

```
1 def fill_node(node, term, pool, min_values, max_values):
2     k = random.randint(min_values, min(max_values, len(pool)))
3     chosen = random.sample(pool, k)
4     if _is_range_datatype(term.get("datatype")):
5         ordered = sorted(chosen, key=lambda v: v["value"])
6         node["value_range"] = {"min": ordered[0]["value"], "max": ordered[-1]["value"]}
7     else:
8         node["values"] = chosen
```

Il numero di valori da usare per un dato vincolo è scelto a caso entro i limiti configurati (listato 5.10), evitando però di selezionare l'intero dominio possibile di un campo, perché un vincolo che copre tutti i valori ammessi non escluderebbe di fatto nessun paziente, risultando così privo di potere selettivo. Se il campo è una data o un valore numerico, i valori scelti vengono usati per costruire un intervallo anziché un elenco discreto.

Listing 5.11: Lo stesso scheletro di listato 5.9, dopo il riempimento: elenco discreto per il campo categoriale, intervallo per quello di tipo data.

```
1 history = [
2     {
3         "query_nodes": [
4             {"predicate": {"label": "gender"}, "values": [{"value": "F", "label": "Female"}], "value_range": None},
5             {"predicate": {"label": "birth date"}, "values": None, "value_range": {"min": "2016-01-01", "max": "2019-01-01"}},
6         ]
7     }
8 ]
```

5.1.8 Generazione del prompt e tracciamento degli span

Definiamo un segmento come un pezzo di testo accompagnato da un booleano che indica se il testo contenga un valore letterale, come una data o un codice, oppure solo una parola di struttura della frase come *with* o *and*. Con lo scheletro completo di valori, il prompt viene costruito ri assemblando i suoi nodi in una lista di segmenti, concatenati uno via l'altro formando il prompt finale. La formulazione di ciascun segmento dipende da cosa contiene il nodo di origine:

- se il nodo ha valori scelti (sezione 5.1.7), il segmento inizia con *with* seguito dai valori stessi, uniti da *or* se sono due, o da una serie di virgole con un *or* finale se sono di più;
- se rappresenta un intervallo, diventa *between X and Y*;
- un nodo senza valori propri, che serve solo a passare al nodo successivo della chain senza introdurre un vincolo, produce invece un segmento con *which* al posto di *with*; un nodo *diagnosis* senza valori seguito da un nodo *stage* con valore *II* produce ad esempio *"which diagnosis with stage II"*.

I segmenti di uno stesso passo della chain vengono uniti con un semplice *and*, e lo stesso connettivo lega tra loro i passi successivi, a partire da un'apertura fissa (*"Find all <entità>"*).

Listing 5.12: Costruzione del prompt segmento per segmento, con tracciamento diretto della posizione di ciascun valore letterale.

```
1 def build_prompt_with_spans(root_label, history, skip_navigation=False):
2     segments = _prompt_segments(root_label, history, skip_navigation)
3     pieces, spans = [], []
4     for is_value, text in segments:
5         if is_value:
6             base = sum(len(p) for p in pieces)
7             spans.append((base, base + len(text)))
8             pieces.append(text)
9     return "".join(pieces), spans
```

Mentre i segmenti vengono concatenati uno alla volta, la posizione di ciascun valore inserito nel testo viene registrata. Determinare gli intervalli solo a partire

dal prompt finale sarebbe infatti ambiguo nel caso in cui lo stesso valore comparisse, per coincidenza, anche in un'altra posizione della frase. Registrare le coordinate durante la costruzione del testo elimina quindi questa ambiguità senza richiedere controlli aggiuntivi. La funzione restituisce sempre la coppia (**prompt**, **value_spans**) coerente tra loro, qualunque sia il testo prodotto; solo più avanti, al momento di salvare il record completo, il flag **include_value_spans** (sezione 5.1.6) decide se il campo **value_spans** venga scritto nell'output finale o scartato, sempre attivo nei run di produzione.

Applicata allo scheletro pieno di listato 5.11, la procedura produce il prompt *"Find all patient with gender Female and birth date between 2016-01-01 and 2019-01-01"*, con **value_spans** pari a [(29, 35), (59, 69), (74, 84)], che corrispondono rispettivamente a *"Female"*, *"2016-01-01"* e *"2019-01-01"*.

5.1.9 Controlli di qualità e deduplicazione

Prima di essere salvata, ogni coppia prompt-query attraversa tre controlli in sequenza.

1. Il primo verifica che la query restituita dall'API contenga davvero una clausola **FILTER** o **VALUES**, oppure almeno uno dei valori letterali scelti. Questo controllo esiste perché l'API potrebbe, in linea di principio, restituire una query sintatticamente valida ma priva di ogni vincolo reale, per esempio se un valore non venisse tradotto correttamente in un filtro effettivo: una query del genere risulterebbe innocua da eseguire, ma completamente slegata dal prompt che dovrebbe rappresentare, e finirebbe nel dataset come esempio scorretto senza che nulla lo segnali. Se nessuna delle due condizioni è soddisfatta, il record viene scartato con il motivo esplicito *"the backend ignored our filters"*.

Listing 5.13: Il primo controllo: verifica che i vincoli scelti siano effettivamente presenti nella query SPARQL restituita.

```
1 def _filters_landed(sparql, chosen_values):
2     if "FILTER" in sparql or "VALUES" in sparql:
3         return True
4     return any(v in sparql for v in chosen_values)
```

2. Il secondo verifica la validità sintattica della query tramite un vero parser SPARQL, la libreria `rdflib`². Serve a intercettare query malformate prima che finiscano nel dataset: un esempio di addestramento con una query che non si può nemmeno interpretare sarebbe peggio che inutile, perché insegnerebbe al modello una sintassi scorretta. Il controllo è condizionato dal parametro `validate_sparql` (listato 5.8), impostato a vero nei run di produzione; ogni query che non superava il parsing veniva quindi scartata per davvero, con il motivo esplicito *"the SPARQL does not parse (rdflib)"*.

Listing 5.14: Il secondo controllo: il parsing sintattico tramite `rdflib` è condizionato dal parametro di configurazione.

```
1 syntax_error = (  
2     validate.sparql_error(sparql)  
3     if self.config.get("validate_sparql", True) else None  
4 )
```

3. Il terzo verifica che la coppia non sia un duplicato, tenendo in memoria l'insieme dei prompt già visti e l'insieme degli hash calcolati su ogni coppia prompt-query già accettata: un nuovo prompt viene scartato se è già in quell'insieme, una nuova coppia se il suo hash coincide con uno già registrato. Senza questo controllo, il campionamento potrebbe scegliere per caso, in record diversi, la stessa combinazione di route e valori, riducendo la varietà effettiva senza che il conteggio totale dei record se ne accorga. In caso di prompt duplicato, il sistema ritenta con nuovi valori scelti a caso fino a 4 volte (`VALUE.REROLLS`) prima di arrendersi.

Solo le coppie che superano tutti e tre i controlli vengono accumulate in un buffer e scritte su disco a gruppi di 50 (`BUFFER_SIZE`).

5.1.10 Generazione manuale nell'interfaccia

Tutto quanto descritto finora in questo capitolo, dalla scoperta della mappa fino ai controlli di qualità, produce record per *una* entità alla volta, data una combinazione specifica di route e proprietà. Il modo più diretto per eseguire questo motore

²<https://rdflib.readthedocs.io/>

è farlo a mano, entità per entità, tramite un pannello dell'interfaccia Streamlit, organizzato in cinque sezioni.

- **Graph Map:** preconditione per tutto il resto, ha un pulsante che permette di caricare la mappa del grafo (sezione 5.1.2) e ne mostra un riepilogo (numero di entità, proprietà e relazioni scoperte) una volta completata.
- **Mode:** sceglie tra questa modalità a entità singola e la generazione automatica da file di tier (sezione 5.1.11).
- **Starting Entity:** un menu a tendina per scegliere l'unica entità radice da generare.
- **Configuration:** quattro slider, uno per relazioni da attraversare, proprietà per percorso, percorsi simultanei e valori per proprietà.
- **Generation Mode e History building:** espongono direttamente, tramite interruttori e caselle di spunta, parametri già discussi altrove in questo capitolo, come la scelta tra campionamento e forza bruta e il numero di worker (sezioni 5.1.4 e 5.1.5), o i flag `offline_history`, `include_value_spans`, `skip_navigation_in_prompt` (sezione 5.1.6).

Gli estremi dei quattro slider di configurazione non sono fissi, ma vengono calcolati al volo scoprendo, tramite `discover_routes_from_map` (sezione 5.1.3), quante route siano effettivamente raggiungibili da quell'entità con i limiti scelti.

Questa modalità non è pensata per costruire un dataset completo, ma per esplorare e calibrare. Al termine della generazione infatti, il pannello non si limita a riportare un conteggio finale, ma mostra due schede separate: una con i record effettivamente creati, in un'anteprima tabellare di prompt e query, e una con quelli scartati, raggruppati per motivo di scarto esatto, tra cui i tre già discussi in sezione 5.1.9 (filtri non atterrati nella query, query sintatticamente non valida, duplicato), con la possibilità di filtrare e ispezionare i singoli tentativi falliti per una singola categoria. È proprio questo riscontro dettagliato a rendere la modalità manuale utile a decidere quali parametri applicare a un tier con decine di entità e tempi di generazione elevati, dove un errore nei parametri rischierebbe di sprecare risorse di calcolo e prolungare inutilmente i tempi di attesa.

5.1.11 Configurazione dei tier da file esterno

Ripetere la procedura manuale appena descritta per ciascuna delle 45 entità dello schema, una alla volta, sarebbe impraticabile, e non lascerebbe alcuna traccia dichiarata e riproducibile delle scelte fatte. Il generatore risolve questo problema raggruppando più entità sotto uno stesso insieme di parametri condivisi, un *tier*: per ciascun tier configurato, per ciascuna entità elencata al suo interno, viene eseguito lo stesso motore di generazione già descritto nelle sottosezioni precedenti, con i parametri propri di quel tier (`min_relations`, `max_relations`, `min_properties`, `max_properties`, `min_paths`, `max_paths`, `min_values`, `max_values`) e un numero massimo di record da generare per quell'entità (`sample_size_per_entity`). I record delle singole entità confluiscono infine in un unico file (sezione 5.1.12).

Per generare il dataset di questa tesi sono state scritte due configurazioni di tier di questo tipo, **TIERS** e **SECOND_TIERS**, create appositamente per questa generazione e non pensate come uno standard riutilizzabile del sistema: al momento in cui sono state scritte, erano liste Python definite direttamente nel codice sorgente, non file esterni. Le due liste raggruppano lo stesso insieme di 45 entità secondo criteri diversi, applicati in due passate di generazione separate. **TIERS** raggruppa le entità per distanza dal nodo **Patient** nel grafo (i sei livelli discussi in capitolo 4); **SECOND_TIERS** raggruppa le stesse entità per tema clinico (*demographics*, *diagnosis-staging*, *treatment*, *measurements-labs*, *symptoms*, *services*). Il commento nel codice sorgente motiva esplicitamente questa scelta: raggruppamenti diversi fanno finire entità diverse nella stessa query multi-relazione, quindi anche un'entità già coperta nella prima passata compare, nella seconda, in combinazioni differenti, ottenendo varietà genuina invece di ricampionare gli stessi pattern di co-occorrenza già visti. Le due configurazioni differiscono anche nei parametri di generazione: mentre in **TIERS** il numero di proprietà vincolate per record varia entro un intervallo (tipicamente 1–3), in **SECOND_TIERS** ogni gruppo impone `min_properties` e `max_properties` uguali tra loro (sempre 4), producendo record strutturalmente più complessi e uniformi rispetto alla prima passata.

Definire una terza configurazione, o modificare le due esistenti, richiedeva a quel tempo di editare direttamente il codice sorgente, senza alcuna verifica automatica prima di avviare la generazione: un errore di battitura o un valore incoerente

si scopriva solo a generazione già avviata, magari dopo ore di chiamate all'API. Per superare questo limite, il generatore accetta oggi anche una configurazione di tier fornita come file JSON esterno, controllata prima di avviare qualunque generazione, senza dover toccare il codice sorgente per definirla o modificarla.

Listing 5.15: Validazione di una configurazione di tier caricata da file esterno, prima di avviare la generazione.

```
1 REQUIRED_TIER_KEYS = {
2     "name", "sample_size_per_entity", "min_relations", "max_relations",
3     "min_paths", "max_paths", "min_properties", "max_properties",
4     "min_values", "max_values", "entities",
5 }
6
7 def validate_tier_config(tiers) -> None:
8     if not isinstance(tiers, list) or not tiers:
9         raise ValueError("expected a non-empty JSON list of tier objects")
10    seen_names = set()
11    for i, tier in enumerate(tiers):
12        if not isinstance(tier, dict):
13            raise ValueError(f"tier #{i + 1} is not an object")
14        missing = REQUIRED_TIER_KEYS - tier.keys()
15        if missing:
16            raise ValueError(f"tier #{i + 1} is missing: {'', '.join(sorted(missing))}")
17        if tier["name"] in seen_names:
18            raise ValueError(f"duplicate tier name: {tier['name']!r}")
19        seen_names.add(tier["name"])
20        if tier["min_relations"] > tier["max_relations"]:
21            raise ValueError(f"tier {tier['name']!r}: min_relations >
22                               max_relations")
23        # ... same check repeated for paths, properties and values
```

La funzione di validazione (listato 5.15) controlla non solo che ogni gruppo contenga tutte le chiavi richieste, ma anche che ogni intervallo dichiarato sia coerente (nessun `min` maggiore del corrispondente `max`) e che non vi siano due gruppi con lo stesso nome, restituendo un messaggio d'errore specifico invece di un'eccezione generica non gestita: un file scritto male viene quindi respinto subito, con un motivo preciso, invece di fallire a metà generazione. La funzione `run_all` (sezione 5.1.5) accetta un parametro `tiers` opzionale per ricevere questa configurazione esterna al posto di quella predefinita.

Questa configurazione esterna è esposta, sul lato dell'interfaccia Streamlit, tramite un pannello dedicato che permette di caricare il file, ne mostra a schermo un

riepilogo (numero di gruppi ed entità coinvolte) prima di abilitare il pulsante di avvio, e traduce l'avanzamento della generazione in una barra di progresso calcolata sul numero di entità completate rispetto al totale. L'output finale non viene scritto in un percorso scelto dall'utente: viene reso disponibile direttamente come file scaricabile al termine del processo, eliminando la necessità di un qualunque accesso al filesystem del server per chi utilizza lo strumento.

5.1.12 Split del dataset

Una volta che tutte le entità dei tier configurati sono state processate e unite in un unico file, l'ultimo passo divide i record in train, validation e test.

Listing 5.16: Suddivisione del dataset in train, validation e test, stratificata per entità di origine.

```
1 for entity, recs in by_entity.items():
2     rng.shuffle(recs)
3     n = len(recs)
4     n_val = max(1, round(n * SPLIT_RATIOS["val"])) if n >= 10 else 0
5     n_test = max(1, round(n * SPLIT_RATIOS["test"])) if n >= 10 else 0
6     n_train = n - n_val - n_test
7     splits["train"].extend(recs[:n_train])
8     splits["val"].extend(recs[n_train:n_train + n_val])
9     splits["test"].extend(recs[n_train + n_val:])
```

Una volta generato l'intero dataset, i record vengono raggruppati per entità di origine e suddivisi in train, validation e test separatamente per ciascun gruppo, con proporzioni 80/10/10 e seme fisso per la riproducibilità (listato 5.16). Le entità con meno di 10 record totali non vengono suddivise: finiscono interamente in train, poiché un campione così piccolo non permetterebbe una validazione o un test statisticamente significativi. Questa stratificazione garantisce che ogni entità sia rappresentata nella stessa proporzione in tutti e tre gli split, evitando che un'entità poco frequente finisca per puro caso quasi interamente in uno solo di essi.

5.2 Implementazione dell'augmentation

5.2.1 Generazione dei placeholder e mascheramento

Il primo passo di ogni record è la sostituzione di ciascun valore letterale con un token opaco (listato 5.17).

Listing 5.17: Generazione dei placeholder e mascheramento del testo a partire dagli span di valore.

```
1  TOKEN_ALPHABET = "ABCDEFGHJKLMNPQRSTUVWXYZ23456789"
2
3  def generate_placeholder_tokens(n):
4      tokens = set()
5      while len(tokens) < n:
6          tokens.add("[VAL_" + "".join(random.choices(TOKEN_ALPHABET, k=4)) + "]")
7      return list(tokens)
8
9  def mask(text, spans):
10     spans = sorted(spans)
11     tokens = generate_placeholder_tokens(len(spans))
12     mapping, pieces, cursor = {}, [], 0
13     for (start, end), placeholder in zip(spans, tokens):
14         pieces.append(text[cursor:start])
15         mapping[placeholder] = text[start:end]
16         pieces.append(placeholder)
17         cursor = end
18     pieces.append(text[cursor:])
19     return "".join(pieces), mapping
```

L'alfabeto dei token esclude deliberatamente i caratteri facilmente confondibili tra loro a colpo d'occhio (I, L, O, 0, 1); i token vengono generati dentro un insieme (`set`) finché non se ne hanno abbastanza di univoci, escludendo così collisioni tra due placeholder nello stesso record. La funzione `mask` opera direttamente sugli span di posizione prodotti dal generatore (sezione 5.1.8), tagliando il testo esattamente su quelle coordinate senza alcuna ricerca a posteriori. Due funzioni di regex ricorrono in tutto il modulo per riconoscere i placeholder, con scopi diversi: una versione rigida (`VAL_RE`), che combacia solo con il formato esatto appena mostrato, usata per estrarre i placeholder autentici da un testo; e una versione permissiva (`LOOSE_VAL_RE`), che riconosce anche forme storpiate o inventate, usata in fase di validazione per intercettare allucinazioni del modello (sezione 5.2.5).

5.2.2 Riformulazione booleana e suggerimenti naturali

Non tutti i placeholder attraversano l'intera pipeline di generazione: i valori booleani vengono eliminati subito, prima ancora di contattare il modello (listato 5.18).

Listing 5.18: Eliminazione anticipata dei placeholder booleani, sostituiti con una forma proposizionale.

```
1 def simplify_boolean_phrasing(text, mapping):
2     new_mapping = dict(mapping)
3     for placeholder, value in mapping.items():
4         if value.lower() not in {"true", "false"}:
5             continue
6         is_true = value.lower() == "true"
7         repl = r"with \1" if is_true else r"without \1"
8         esc = re.escape(placeholder)
9         stop = r"(?:with|which|and)"
10        which_re = re.compile(rf"\bwhich ((?:?!{stop}\b)\w+ ?){1,5}} is {esc}",
11                               re.IGNORECASE)
12        with_re = re.compile(rf"\bwith ((?:?!{stop}\b)\w+ ?){1,5}} {esc}", re.
13                               IGNORECASE)
14        if which_re.search(text):
15            text = which_re.sub(repl, text)
16            del new_mapping[placeholder]
17        elif with_re.search(text):
18            text = with_re.sub(repl, text)
19            del new_mapping[placeholder]
20    return text, new_mapping
```

Un placeholder che rappresenta un valore vero viene sostituito con *"with"* seguito dal termine clinico associato, uno falso con *"without"*; in entrambi i casi il placeholder viene rimosso anche dalla mappa restituita, non solo dal testo, così il resto della pipeline non si aspetta più di doverlo ritrovare. Per tutti gli altri valori mascherati, una funzione separata (`build_hint_map`) prepara un suggerimento in linguaggio naturale, in ordine di priorità: data ISO trasformata in forma scritta per esteso, numero piccolo (fino a 999) trasformato in lettere, etichetta composta con trattino ridotta alla sola parte specifica; solo se nessuna di queste regole deterministiche si applica, viene interrogato il modello per un sinonimo d'uso comune. Questo suggerimento viene usato, con una probabilità configurabile, al momento del ripristino finale dei valori (sezione 5.2.5), e in modo differente nella modalità priva di mascheramento (sezione 5.2.8).

5.2.3 Suddivisione in chunk per record complessi

Un record con molti valori mascherati viene spezzato in più porzioni prima di essere sottoposto al modello, per evitare il degrado di qualità osservato empiricamente oltre una certa densità di placeholder in una singola chiamata.

Listing 5.19: Suddivisione del testo mascherato in chunk, rispettando il vincolo sugli intervalli *between...and...*

```
1 MAX_VALUES_PER_CHUNK = 12
2
3 def split_into_chunks(masked_text, mapping, max_values_per_chunk):
4     clauses = _protect_between(masked_text).split(CLAUSE_SEP)
5     chunks = [[]]
6     count_in_chunk = 0
7     for clause in clauses:
8         n_values = len(VA_RE.findall(clause))
9         if count_in_chunk > 0 and count_in_chunk + n_values > max_values_per_chunk:
10             :
11             chunks.append([])
12             count_in_chunk = 0
13             chunks[-1].append(clause)
14             count_in_chunk += n_values
15     return [_restore_between(CLAUSE_SEP.join(c)) for c in chunks if c]
```

Il testo viene diviso in clausole separate dalla congiunzione *"and"*, accumulate in un chunk finché non si supererebbe la soglia di 12 valori (`MAX_VALUES_PER_CHUNK`); un meccanismo dedicato (`_protect_between/_restore_between`) sostituisce temporaneamente l'*"and"* interno a un intervallo *"between X and Y"* con un segnaposto interno che non compare mai per caso in un testo inglese (il carattere NUL), impedendo che un intervallo venga spezzato a metà tra due chunk diversi. Ogni record con un numero di valori pari o inferiore alla soglia produce un solo chunk, e non attraversa quindi alcuna suddivisione: nel dataset supplementare da 1331 record generato con vincolo fisso a 4 proprietà per record (sezione 5.1.11), l'80,6% dei record (1073 su 1331) è rimasto in un solo chunk, mentre il restante 19,4% (258 record) è stato diviso in due; nessun record ha richiesto più di due chunk, nemmeno quello con più valori in assoluto (20 valori, divisi in due chunk da 11 e 9).

5.2.4 Generazione iterativa: stile, retry e fallback

Ogni chunk, e successivamente ogni variante completa, viene generato tramite un ciclo di tentativi con feedback esplicito sul motivo di un eventuale rifiuto e temperatura crescente (listato 5.20).

Listing 5.20: Ciclo di generazione e validazione per un singolo chunk, con fallback esplicito in caso di fallimento persistente.

```
1 def generate_piece(model, piece_text, piece_mapping, style_desc,
2                     max_attempts, base_temp, max_temp,
3                     avoid_history, indent, label,
4                     avoid_text=None, avoid_reason=None,
5                     as_fragment=False, language="English"):
6     total_elapsed = 0.0
7     for attempt in range(max_attempts):
8         temperature = temperature_for_attempt(attempt, max_attempts, base_temp,
9                                                max_temp)
9         candidate, elapsed = (
10             paraphrase_fragment(model, piece_text, piece_mapping, temperature,
11                                language, avoid_text, avoid_reason)
12             if as_fragment else
13             paraphrase_one(model, piece_text, piece_mapping, style_desc,
14                            temperature, language,
15                            avoid_text, avoid_reason, avoid_history if attempt ==
16                               0 else None)
17         )
18         total_elapsed += elapsed
19         reason = validation_reason(candidate, piece_mapping, [])
20         if reason is None:
21             return candidate, total_elapsed, False
22         avoid_text, avoid_reason = candidate, reason
23     return piece_text, total_elapsed, True
```

Se il candidato viene rifiutato, il testo scartato e il motivo esatto del rifiuto diventano il contesto per il tentativo successivo, mentre la temperatura sale linearmente dalla base al massimo configurato (`temperature_for_attempt`). Se tutti i tentativi (5 di default) falliscono, la funzione non forza comunque un output: restituisce il testo originale non modificato, segnalando esplicitamente il ricorso a questo comportamento di ripiego. Il primo chunk di un record usa il template completo, con l'assegnazione di stile e la cronologia delle aperture di frase recenti da evitare; i chunk successivi usano un template più semplice *"a frammento"* (`as_fragment=True`), senza ripetere l'assegnazione di stile, per restare coerenti col registro già stabilito dal primo pezzo. Ogni chunk viene validato singolarmente.

te, ma solo sull'integrità dei placeholder: il controllo di somiglianza, che richiede un confronto con varianti già accettate, si applica soltanto al termine, sul testo ricomposto (sezione 5.2.5).

5.2.5 Validazione: due funzioni per due modalità

La validazione di un candidato applica fino a tre criteri in sequenza all'interno di un'unica funzione (listato 5.21), più un quarto controllo esterno sulla ripetizione dell'apertura di frase.

Listing 5.21: La funzione di validazione condivisa da chunk e varianti complete, con soglia di somiglianza adattiva.

```
1 def validation_reason(candidate, mapping, accepted_so_far, max_similarity=None):
2     missing = [p for p in mapping if p not in candidate]
3     if missing:
4         return f"you dropped the placeholder(s) {'', '.join(missing)} - every
5             single one must appear unchanged"
6
7     malformed = sorted(set(m for m in LOOSE_VAL_RE.findall(candidate) if m not in
8         mapping))
9     if malformed:
10        return f"you wrote malformed or invented placeholder-like text: {'', '.join
11            (malformed)} - placeholders must be copied exactly, never altered"
12
13    threshold = max_similarity if max_similarity is not None else
14        similarity_threshold_for(candidate)
15    for prev in accepted_so_far:
16        ratio = difflib.SequenceMatcher(None, candidate.lower(), prev.lower()).
17            ratio()
18        if ratio >= threshold:
19            return f"this wording is too similar ({ratio:.0%} match) to a variant
20                already produced for this same request"
21    return None
```

Il controllo di somiglianza usa `difflib.SequenceMatcher`³ della libreria standard, con una soglia che dipende dalla lunghezza del testo: più permissiva (fino al 97%) per frasi brevi, più severa (85%) per frasi lunghe, altrimenti testi corti verrebbero quasi sempre scartati come duplicati per mancanza di margine di variazione lessicale. Poiché questa funzione riceve sempre una lista vuota quando

³<https://docs.python.org/3/library/difflib.html>

è invocata sui singoli chunk, il controllo di somiglianza non si attiva mai a quel livello: è un ciclo su una lista vuota, non un controllo disattivato esplicitamente.

Per la modalità priva di mascheramento (sezione 5.2.8) esiste una seconda funzione di validazione, più permissiva sul primo controllo: un valore non è considerato perso se compare nella sua forma di suggerimento naturale, o come sinonimo plausibile case-insensitive, non solo nella sua forma letterale esatta, coerentemente col fatto che, in questa modalità, il modello vede il valore vero e ha il permesso esplicito di riformularlo.

5.2.6 Rotazione degli stili e memoria anti-ripetizione

La varietà stilistica tra le varianti di uno stesso record è governata da un oggetto che assegna gli stili in sequenza fissa e ricorda le generazioni recenti per stile (listato 5.22).

Listing 5.22: Assegnazione ciclica degli stili e memoria a finestra delle generazioni recenti.

```
1 class StyleTracker:
2     def __init__(self, styles=None, recent_size=3, max_chars=160, opener_window
3         =12):
4         self._styles = styles if styles is not None else STYLES
5         self._style_cycle = itertools.cycle(self._styles)
6         self._recent_texts = {tag: [] for tag, _ in self._styles}
7         self._opener_history = {tag: [] for tag, _ in self._styles}
8
9     def next_style(self):
10         return next(self._style_cycle)
11
12     def remember(self, style_tag, text):
13         sanitized = VAL_RE.sub("...", text)
14         preview = sanitized if len(sanitized) <= self._max_chars else sanitized[:
15             self._max_chars - 3] + "..."
16         bucket = self._recent_texts.setdefault(style_tag, [])
17         bucket.append(preview)
18         del bucket[: -self._recent_size]
19         openers = self._opener_history.setdefault(style_tag, [])
20         openers.append(opener(text))
21         del openers[: -self._opener_window]
```

Gli stili (dieci registri predefiniti, da tecnico-database a colloquiale a statistico) vengono assegnati percorrendo un ciclo fisso (`itertools.cycle`⁴), non estratti a caso, per garantire una distribuzione uniforme su un numero elevato di record. `remember` viene chiamata solo su varianti accettate, mai su tentativi rifiutati: tiene una finestra delle ultime 3 formulazioni complete per stile (con i placeholder ancora presenti sostituiti da "...", per non confrontare mai codici opachi come se fossero testo naturale) e una finestra più ampia, le ultime 12, delle sole aperture di frase. La finestra qui è più ampia perché le prime due parole di una frase sono un'informazione più povera di una frase intera, e richiedono quindi uno storico più esteso per intercettare ripetizioni. Le due informazioni sono tenute in due dizionari distinti, invece che in un'unica struttura combinata, proprio perché servono controlli diversi con finestre di lunghezza diversa: tenerle separate evita di dover troncare artificialmente l'una alla lunghezza dell'altra.

5.2.7 Statistiche di qualità e resilienza

Ogni run accumula un insieme di contatori che permettono, a posteriori, di distinguere una variante uscita pulita da una uscita con un fallback, da una richiesta di variante che non ha prodotto nulla.

Listing 5.23: Calcolo della resa complessiva di un run, a partire dagli slot totali e da quelli completamente falliti.

```
1 def to_dict(self):
2     slots_failed = len(self.failed_slots)
3     yield_rate = (1 - (slots_failed / self.total_slots)) if self.total_slots else
4         None
5     return {
6         "total_slots": self.total_slots,
7         "slots_failed": slots_failed,
8         "yield_rate": yield_rate,
9         "fallback_count": self.fallback_count,
10        "style_attempts": dict(self.style_attempts),
11        "style_accepted": dict(self.style_accepted),
12        "failed_slots": self.failed_slots,
13    }
```

`fallback_count` conta le varianti uscite comunque, ma con almeno un chunk non riformulato; `failed_slots` registra invece le richieste di variante per cui, dopo

⁴<https://docs.python.org/3/library/itertools.html#itertools.cycle>

aver esaurito i tentativi esterni di rigenerazione completa (non solo i tentativi per singolo chunk), non si è prodotto nulla; per ciascuna, viene conservato anche il dettaglio di ogni tentativo fallito e il relativo motivo. Le statistiche vengono salvate su disco dopo ogni record processato, non solo a fine run: un'esecuzione interrotta e ripresa continua ad accumulare sugli stessi contatori invece di ripartire da zero. Allo stesso scopo di resilienza, l'elenco dei `source_id` già presenti nel file di output viene ricostruito a ogni avvio, così che i record già completati vengano saltati in caso di ripresa dopo un'interruzione (*RNF2*).

5.2.8 La modalità senza mascheramento

Il sistema conserva, per ciascun record, anche un percorso di generazione alternativo che riproduce il comportamento pre-mascheramento: il testo intero viene sottoposto al modello con i valori veri elencati esplicitamente, senza alcuna suddivisione in chunk (listato 5.24).

Listing 5.24: La modalità senza mascheramento, mantenuta come termine di paragone esplicito e non come percorso di produzione alternativo.

```
1 def generate_variant_no_mask(model, text, values, style_tag, style_desc,
2                             max_attempts, base_temp, max_temp,
3                             avoid_history, slot_label,
4                             avoid_text=None, avoid_reason=None, language="
5                             English", hints=None):
6
7     # Deliberately no chunking here: this mode exists to reproduce the
8     # pre-masking, single-shot behavior for direct comparison, not to be a
9     # second production path.
10    hints = hints or {}
11    identity_mapping = {v: v for v in values}
12    value_list = ", ".join(f'"{v}"' for v in values)
13    ...
14    reason = validation_reason_with_hints(candidate, values, hints, [])
```

Il commento nel codice sorgente è esplicito sul ruolo di questa modalità: non un secondo percorso di produzione, ma un termine di paragone diretto per quantificare in valutazione il beneficio del mascheramento (capitolo 6). L'evidenza raccolta in un run diagnostico su questa modalità conferma empiricamente la scelta progettuale discussa in capitolo 4: su 41 rifiuti registrati nel log di un run limitato a due record, 36 (87,8%) erano dovuti a valori letterali persi o alterati dal modello:

un esito reso strutturalmente impossibile dal mascheramento, dato che in quella modalità il modello non ha mai accesso al valore reale durante la riformulazione.

Capitolo 6

Valutazione

Capitolo 7

Conclusioni e sviluppi futuri

Acknowledgements

la vuoi